Lab-1

Aim: write a program to convert infix to postfix

Resource used: online GDB C compiler.

Code:

```
#include<stdio.h>
#include<ctype.h>

char stack[100];
int top = -1;

void push(char x)
{
    stack[++top] = x;
}

char pop()
{
    if(top == -1)
        return -1;
    else
        return stack[top--];
}

int priority(char x)
{
    if(x == '(')
```

```
        return 0;
    if(x == '+' || x == '-')
        return 1;
    if(x == '*' || x == '/')
        return 2;
    return 0;
}

int main()
{
    char exp[100];
    char *e, x;
    printf("Enter the expression : ");
    scanf("%s",exp);
    printf("\n");
    e = exp;

    while(*e != '\0')
    {
        if(isalnum(*e))
            printf("%c ",*e);
        else if(*e == '(')
            push(*e);
        else if(*e == ')')
        {
            while((x = pop()) != '(')
                printf("%c ", x);
        }
```

```
        else
        {
            while(priority(stack[top]) >= priority(*e))
                printf("%c ",pop());
            push(*e);
        }
        e++;
    }
    while(top != -1)
    {
        printf("%c ",pop());
    }return 0;
}
```

Output:

```
Enter the expression : A+B*C+D

A B C * + D +

...Program finished with exit code 0
Press ENTER to exit console.
```

Lab-2A

Aim: write a program to implement the regular expression to DFA for 0*1*(0+1)

Resource used: online GDB C compiler.

Code:

```
#include<stdio.h>
#include<conio.h>
#include<strings.h>
void main() {
  int table[2][2],i,j,l,status=0,success;
  char input[100];
  printf("To implementing DFA of language 0*1*(0+1)\n");
  printf("\n\nEnter Input String:");
  table[0][0]=1;
  table[0][1]=-1;
  table[1][0]=1;
  table[1][1]=0;
  scanf("%s",input);
  l=strlen(input);
  for (i=0;i<l;i++) {
    if(input[i]!='0'&&input[i]!='1') {
      printf("The entered Value is wrong");
      getch();
      exit(0);
    }
    if(input[i]=='0')
    status=table[status][0];
```

```
    else
    status=table[status][1];
    if(status==-1) {
      printf("String not Accepted");
      break;
    }
  }
  if(i==l)
    printf("String Accepted");
  getch();
}
```

Output:

```
To implementing DFA of language 0*1*(0+1)


Enter Input String:01010
String Accepted
```

Lab-2B

Aim: write a program to implement the regular expression to DFA for a+b*(c+d)+a*c+

Resource used: online GDB C compiler.

Code:

```
#include<stdio.h>

#include<conio.h>

#include<strings.h>

void main() {

  int table[2][2],i,j,l,status=0,success;

  char input[100];

  printf("To implementing DFA of language a+b*(c+d)+a*c+)");

  printf("\n\nEnter Input String:");

  table[0][0]=1;

  table[0][1]=-1;

  table[1][0]=1;

  table[1][1]=0;

  scanf("%s",input);

  l=strlen(input);

  for (i=0;i<l;i++) {

    if(input[i]!='a'&&input[i]!='b' &&input[i]!='c') {

      printf("The entered Value is wrong");

      getch();
```

```
        exit(0);
      }
      if(input[i]=='a')
      status=table[status][0];
      else
      status=table[status][1];
      if(status==-1) {
        printf("String not Accepted");
        break;
      }
    }
    if(i==l)
      printf("String Accepted");
    getch();
}
```

Output:

```
To implementing DFA of language a+b*(c+d)+a*c+

Enter Input String:aaba
String Accepted

...Program finished with exit code 0
Press ENTER to exit console.
```

Lab-3A

Aim: Implement a program to find the tokens for a expression

Resource Used: online GDB C compiler.

Code:

```
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
bool isValidDelimiter(char ch) {
   if (ch == ' ' || ch == '+' || ch == '-' || ch == '*' ||
   ch == '/' || ch == ',' || ch == ';' || ch == '>' ||
   ch == '<' || ch == '=' || ch == '(' || ch == ')' ||
   ch == '[' || ch == ']' || ch == '{' || ch == '}')
   return (true);
   return (false);
}
bool isValidOperator(char ch){
   if (ch == '+' || ch == '-' || ch == '*' ||
   ch == '/' || ch == '>' || ch == '<' ||
   ch == '=')
   return (true);
   return (false);
}
// Returns 'true' if the string is a VALID IDENTIFIER.
bool isvalidIdentifier(char* str){
   if (str[0] == '0' || str[0] == '1' || str[0] == '2' ||
   str[0] == '3' || str[0] == '4' || str[0] == '5' ||
```

```
   str[0] == '6' || str[0] == '7' || str[0] == '8' ||
   str[0] == '9' || isValidDelimiter(str[0]) == true)
   return (false);
   return (true);
}
bool isValidKeyword(char* str) {
   if (!strcmp(str, "if") || !strcmp(str, "else") || !strcmp(str, "while") || !strcmp(str, "do") ||
!strcmp(str, "break") || !strcmp(str, "continue") || !strcmp(str, "int")
   || !strcmp(str, "double") || !strcmp(str, "float") || !strcmp(str, "return") || !strcmp(str,    "char") ||
!strcmp(str, "case") || !strcmp(str, "char")
   || !strcmp(str, "sizeof") || !strcmp(str, "long") || !strcmp(str, "short") || !strcmp(str, "typedef") ||
!strcmp(str, "switch") || !strcmp(str, "unsigned")
   || !strcmp(str, "void") || !strcmp(str, "static") || !strcmp(str, "struct") || !strcmp(str, "goto"))
   return (true);
   return (false);
}
bool isValidInteger(char* str) {
   int i, len = strlen(str);
   if (len == 0)
   return (false);
   for (i = 0; i < len; i++) {
      if (str[i] != '0' && str[i] != '1' && str[i] != '2'&& str[i] != '3' && str[i] != '4' && str[i] != '5'
      && str[i] != '6' && str[i] != '7' && str[i] != '8' && str[i] != '9' || (str[i] == '-' && i > 0))
      return (false);
   }
   return (true);
}
bool isRealNumber(char* str) {
   int i, len = strlen(str);
```

```
   bool hasDecimal = false;
   if (len == 0)
   return (false);
   for (i = 0; i < len; i++) {
      if (str[i] != '0' && str[i] != '1' && str[i] != '2' && str[i] != '3' && str[i] != '4' && str[i]       !=
'5' && str[i] != '6' && str[i] != '7' && str[i] != '8'
      && str[i] != '9' && str[i] != '.' || (str[i] == '-' && i > 0))
      return (false);
         if (str[i] == '.')
      hasDecimal = true;
   }
   return (hasDecimal);
}
char* subString(char* str, int left, int right) {
   int i;
   char* subStr = (char*)malloc( sizeof(char) * (right - left + 2));
   for (i = left; i <= right; i++)
      subStr[i - left] = str[i];
   subStr[right - left + 1] = '\0';
   return (subStr);
}
void detectTokens(char* str) {
   int left = 0, right = 0;
   int length = strlen(str);
   while (right <= length && left <= right) {
      if (isValidDelimiter(str[right]) == false)
      right++;
      if (isValidDelimiter(str[right]) == true && left == right) {
         if (isValidOperator(str[right]) == true)
```

```
        printf("\nValid operator : '%c'", str[right]);
        right++;
        left = right;
      } else if (isValidDelimiter(str[right]) == true && left != right || (right == length && left !=
right)) {
        char* subStr = subString(str, left, right - 1);
        if (isValidKeyword(subStr) == true)
           printf("\nValid keyword : '%s'", subStr);
        else if (isValidInteger(subStr) == true)
           printf("\nValid Integer : '%s'", subStr);
        else if (isRealNumber(subStr) == true)
           printf("\nReal Number : '%s'", subStr);
        else if (isvalidIdentifier(subStr) == true
           && isValidDelimiter(str[right - 1]) == false)
        printf("\nValid Identifier : '%s'", subStr);
        else if (isvalidIdentifier(subStr) == false
           && isValidDelimiter(str[right - 1]) == false)
        printf("\nInvalid Identifier : '%s'", subStr);
        left = right;
      }
   }
   return;
}
int main(){
   char str[100];
   printf("Enter the token expression:");
   scanf("%s", str);
   printf("All Tokens are : ");
   detectTokens(str);
```

```
  return (0);
}
```

Output:

```
Enter the token expression:a+b*int-0+true
All Tokens are :
Valid Identifier : 'a'
Valid operator : '+'
Valid Identifier : 'b'
Valid operator : '*'
Valid keyword : 'int'
Valid operator : '-'
Valid Integer : '0'
Valid operator : '+'
Valid Identifier : 'true'

...Program finished with exit code 0
Press ENTER to exit console.
```

Table of Content

S.No	Practical	Date Assigned	Signature
1.	Write a program to convert infix to postfix	03-01-2023	
2.	Consider the following regular expressions: a) 0*1*(0+1) b) (a*bc +a*d+e)+ c) ((a +b)*(c + d)*)+ abc*d* Write separate programs for each of the regular expressions mentioned above.	17-01-2023	
3.	a. Design a Lexical analyzer for identifying different types of token and show the output for the given string a=b+c*d b. Write a program which accepts a regular expression from the user and generates a regular grammar which is equivalent to the R.E. entered by user. The grammar will be printed to a text file, with only one production rule in each line. Also, make sure that all production rules are displayed in compact forms e.g. the production rules: S--> aB, S--> cd S- -> PQ Should be written as S--> aB | cd | PQ And not as three different production rules. Also, there should not be any repetition of production rules.	24-01-2023	
4.	Write a program to count the number tokens in a file	31-01-2023	
5.	a.write a program to convert regular expression to Context free grammar b. Write a program for Recursive Descent parsing.		
6.	a. Write a program to eliminate left recursion for any given grammar.		

	b. To Compute FIRST () and FOLLOW of the desired Non-Terminal Element of given Grammar.		
7.	Consider the following grammar: S --> ABC A--> abA | ab B--> b | BC C--> c | cC Following any suitable parsing technique(prefer topdown), design a parser which accepts a string and tells whether the string is accepted by above grammar or not		
8.	To Compute LEADING() and TRAILING() of the desired Non-Terminal Element of given Grammar.		
9.	Write a program which accepts a regular grammar with no left recursion, and no nullproduction rules, and then it accepts a string and reports whether the string is accepted by the grammar or not.		
10.	Design a parser which accepts a mathematical expression (containing integers only). If the expression is valid, then evaluate the expression else report that the expression is invalid. [Note: Design first the Grammar and then implement using ShiftReduce parsing technique. Your program should generate an output file clearly showing each step of parsing/evaluation of the intermediate subexpressions.		

Lab-5B

Date: / /2023

Aim: Write a program for Recursive Descent parsing.

Resource used: online GDB C Compiler

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

char input[100];
int i = 0;

void error() {
    printf("Invalid input!\n");
    exit(1);
}

void expr();
void term();
void factor();
void number();

void expr() {
    term();
    while (input[i] == '+' || input[i] == '-') {
        i++;
        term();
```

```
    }
}

void term() {
    factor();
    while (input[i] == '*' || input[i] == '/') {
        i++;
        factor();
    }
}
void factor() {
    if (isdigit(input[i])) {
        number();
    }
    else if (input[i] == '(') {
        i++;
        expr();
        if (input[i] == ')') {
            i++;
        }
        else {
            error();
        }
    }
    else {
        error();
    }
}
void number() {
    while (isdigit(input[i])) {
```

```
        i++;
    }
}
int main() {
    printf("Enter an arithmetic expression: ");
    fgets(input, 100, stdin);
    input[strcspn(input, "\n")] = 0;
    expr();
    if (i == strlen(input)) {
        printf("Valid input!\n");
    }
    else {
        error();
    }
    return 0;
}
```

Output:

Lab-3B

Date: / /2023

Aim: Write a program which accepts a regular expression from the user and generates a regular grammar which is equivalent to the R.E. entered by user. The grammar will be printed to a text file, with only one production rule in each line. Also, make sure that all production rules are displayed in compact forms e.g. the production rules: S--> aB, S--> cd S- -> PQ Should be written as S--> aB | cd | PQ And not as three different production rules. Also, there should not be any repetition of production rules

Resource Used: Online GDB C Compiler

Code:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>


#define MAX_PRODUCTIONS 100

#define MAX_SYMBOLS 100

int production_count = 0;

char productions[MAX_PRODUCTIONS][MAX_SYMBOLS];

void add_production(const char *lhs, const char *rhs)

{

    char production[MAX_SYMBOLS];

    snprintf(production, MAX_SYMBOLS, "%s%s", lhs, rhs);

    for (int i = 0; i < production_count; i++) {

        if (strcmp(production, productions[i]) == 0) {

            return; // already exists

        }

    }

    strcpy(productions[production_count], production);

    production_count++;

}
```

```
void generate_grammar(const char *regex, const char *symbol)
{
    if (*regex == '\0') {
        add_production(symbol, ""); // empty string
        return;
    }
    char first = *regex;
    char rest[MAX_SYMBOLS];
    strcpy(rest, regex + 1);
    if (rest == '*') {
        // A* -> A A*
        char non_terminal[2] = {0};
        snprintf(non_terminal, 2, "%d", production_count);
        add_production(symbol, non_terminal);
        add_production(non_terminal, ""); // empty string
        generate_grammar(rest + 1, non_terminal);
        add_production(non_terminal, non_terminal);
    } else {
        // A -> a A' | a
        char non_terminal[2] = {0};
        snprintf(non_terminal, 2, "%d", production_count);
        char terminal[2] = {0};
        snprintf(terminal, 2, "%c", first);
        add_production(symbol, terminal);
        generate_grammar(rest, non_terminal);
        add_production(non_terminal, ""); // empty string
        add_production(symbol, non_terminal);
    }
}
int main()
{
```

```
    char regex[MAX_SYMBOLS];
    printf("Enter a regular expression: ");
    scanf("%s", regex);
    generate_grammar(regex, "S");
    FILE *file = fopen("grammar.txt", "w");
    for (int i = 0; i < production_count; i++) {
        fprintf(file, "%s", productions[i]);
        if (i < production_count - 1) {
            fprintf(file, " | ");
        }
    }
    fclose(file);
    printf("Grammar generated and written to grammar.txt\n");
    return 0;
}
```

Output:

```
main.c: In function 'generate_grammar':
main.c:41:14: warning: comparison between pointer and integer
   41 |     if (rest == '*') {
      |              ^~
Enter a regular expression: 2+3*(4-1)
Grammar generated and written to grammar.txt


...Program finished with exit code 0
Press ENTER to exit console.
```

Lab-4

Date:31/01/2023

Aim: To count the number of tokens in a program.

Resource used: Online GDB C++ compiler.

Code:

```
#include <iostream>
#include <fstream>
#include <regex>

using namespace std;

int main() {
    string filename;
    cout << "Enter file name: ";
    getline(cin, filename);

    ifstream file(filename);
    string line;
    int token_count = 0;
    if (file.is_open()) {
        while (getline(file, line)) {
            if (line.find("#include") != string::npos) {
                continue;
            }
            if (line[0] == '#') {
                continue;
            }
            if(line.find("//") != string::npos){
                continue;
            }
            if(line.find("/*") != string::npos){
                continue;
            }
            if(line.find("*/") != string::npos){
                continue;
            }
            if(line.find("::") != string::npos){
                token_count--;
            }
            if(line.find(">>") != string::npos){
                token_count--;
            }
            if(line.find("<<") != string::npos){
                token_count--;
            }
            if(line.find("->") != string::npos){
```

```
    token_count--;
}
if(line.find("++") != string::npos){
    token_count--;
}
if(line.find("--") != string::npos){
    token_count--;
}
if(line.find("==") != string::npos){
    token_count--;
}
if(line.find("!=") != string::npos){
    token_count--;
}
if(line.find("<=") != string::npos){
    token_count--;
}
if(line.find(">=") != string::npos){
    token_count--;
}
if(line.find("&&") != string::npos){
    token_count--;
}
if(line.find("||") != string::npos){
    token_count--;
}
if(line.find("+=") != string::npos){
    token_count--;
}
if(line.find("-=") != string::npos){
    token_count--;
}
if(line.find("*=") != string::npos){
    token_count--;
}
if(line.find("/=") != string::npos){
    token_count--;
}
if(line.find("%=") != string::npos){
    token_count--;
}
if(line.find("&=") != string::npos){
    token_count--;
}
if(line.find("|=") != string::npos){
    token_count--;
}
if(line.find("^=") != string::npos){
    token_count--;
}
```

```
        if(line.find("<<=") != string::npos){
            token_count--;
        }
        if(line.find(">>=") != string::npos){
            token_count--;
        }

        regex token_regex("[a-zA-Z_][a-zA-Z_0-9]*|[0-9]+|[(){};,::]|[+\\-*/=<>]|[^ \t\r\n]]");
        sregex_iterator iter(line.begin(), line.end(), token_regex);
        sregex_iterator end;
        for (; iter != end; ++iter) {
            string token = iter->str();
            token_count ++;
        }
    }
    file.close();
    cout << "Number of tokens: " << token_count << endl;
  }
  else {
    cout << "Unable to open file" << endl;
  }
  return 0;
}
```

Output:

```
Enter file name: jaswanth.txt
Number of tokens: 20


...Program finished with exit code 0
Press ENTER to exit console.
```

Lab-5

Date: / / 2023

Aim: Write a program to convert Regular Expression to Context Free Grammar (CFG)

Resource Used: Online GDB C++ compiler.

Code:

```
#include<iostream>
#include<stdlib.h>
#include<string.h>
using namespace std;
int main () {
string regex="";
cout<<"Enter the regex: ";
cin>> regex;
int len = regex.length();
cout<<"CFG: "<<endl;
for(int i=0;i<len;i++){
if(regex[i]=='*')
cout<<"S-> "<<regex[i-1]<<"S | epsilon"<<endl;
if(regex[i]=='|')
cout<<"S-> "<<regex[i-1]<<" | "<<regex[i+1]<<endl;
if(regex[i]=='+' && regex[i+1]!='*')
cout<<"S-> "<<regex[i-1]<<"S | "<<regex[i-1]<<endl;
if(regex[i]=='.')
cout<<"S-> "<<regex[i-1]<<regex[i+1]<<endl;}
return 0;}
```

Output:

```
Enter the regex: a*b*c
CFG:
S-> aS | epsilon
S-> bS | epsilon


...Program finished with exit code 0
```

P V S JASWANTH
A2305220687

LAB-6 A

Date: 14/02/2023

Aim: Write a program to remove left recursion.

CODE:

```
#include<stdio.h>
#include<string.h>
#define SIZE 10
  int main () {
    char non_terminal;
    char beta,alpha;
    int num;
    char production[10][SIZE];
    int index=3; /* starting of the string following "->" */
    printf("Enter Number of Production : ");
    scanf("%d",&num);
    printf("Enter the grammar as E->E-A :\n");
    for(int i=0;i<num;i++){
        scanf("%s",production[i]);
    }
    for(int i=0;i<num;i++){
        printf("\nGRAMMAR : : : %s",production[i]);
        non_terminal=production[i][0];
        if(non_terminal==production[i][index]) {
            alpha=production[i][index+1];
        printf(" is left recursive.\n");
        while(production[i][index]!=0 && production[i][index]!='|')
            index++;
        if(production[i][index]!=0) {
            beta=production[i][index+1];
            printf("Grammar without left recursion:\n");
            printf("%c->%c%c\'",non_terminal,beta,non_terminal);
            printf("\n%c\'->%c%c\'|E\n",non_terminal,alpha,non_terminal);
        }

else
            printf(" can't be reduced\n");
    }
    else
        printf(" is not left recursive.\n");
    index=3;
    }
}
```

P V S Jaswanth Kumar
A2305220687
6CSE-2 Y

Output:

```
Enter Number of Production : 4
Enter the grammar as E->E-A :
E->EA|A
A->AT|a
T->a
E->i

GRAMMAR : : : E->EA|A is left recursive.
Grammar without left recursion:
E->AE'
E'->AE'|E

GRAMMAR : : : A->AT|a is left recursive.
Grammar without left recursion:
A->aA'
A'->TA'|E

GRAMMAR : : : T->a is not left recursive.

GRAMMAR : : : E->i is not left recursive.


...Program finished with exit code 0
Press ENTER to exit console.
```

LAB – 6 B

Date: 14/02/2023

Aim: To Compute First () And Follow Of The Desired Non-Terminal Element Of Given Grammar.

Code:

```
#include <iostream>

#include <unordered_map>

#include <vector>

#include <set>


using namespace std;


unordered_map<char, vector<string>> grammar = {

    {'S', {"AB", "BC"}},

    {'A', {"aB", "ab"}},

    {'B', {"b", "BC"}},

    {'C', {"cC", "c"}}

};


// Compute the first set of a given non-terminal symbol

set<char> compute_first(char symbol) {

    set<char> first_set;

    for (const auto &production : grammar[symbol]) {

        if (isupper(production[0])) {

            set<char> sub_first_set = compute_first(production[0]);

            first_set.insert(sub_first_set.begin(), sub_first_set.end());

        } else {

            first_set.insert(production[0]);

        }

    }

    return first_set;
```

```
}

// Compute the follow set of a given non-terminal symbol
set<char> compute_follow(char symbol) {
    set<char> follow_set;
    if (symbol == 'S') {
        follow_set.insert('$');
    }
    for (const auto &kv : grammar) {
        for (const auto &production : kv.second) {
            for (int i = 0; i < production.length(); ++i) {
                if (production[i] == symbol) {
                    if (i == production.length() - 1) {
                        if (kv.first != symbol) {
                            set<char> sub_follow_set = compute_follow(kv.first);
                            follow_set.insert(sub_follow_set.begin(), sub_follow_set.end());
                        }
                    } else {
                        if (isupper(production[i + 1])) {
                            set<char> sub_first_set = compute_first(production[i + 1]);
                            if (sub_first_set.find('e') != sub_first_set.end()) {
                                sub_first_set.erase('e');
                                set<char> sub_follow_set = compute_follow(kv.first);
                                sub_first_set.insert(sub_follow_set.begin(), sub_follow_set.end());
                                follow_set.insert(sub_first_set.begin(), sub_first_set.end());
                            } else {
                                follow_set.insert(sub_first_set.begin(), sub_first_set.end());
                            }
                        } else {
                            follow_set.insert(production[i + 1]);
```

```
                }
            }
        }
    }
    }}
    return follow_set;
}
int main() {
    // Compute and output the first and follow sets of the non-terminals
    for (const auto &kv : grammar) {
        cout << "First(" << kv.first << ") = { ";
        set<char> first_set = compute_first(kv.first);
        for (char symbol : first_set) {
            cout << symbol << " ";
        }
        cout << "}" << endl;
        cout << "Follow(" << kv.first << ") = { ";
        set<char> follow_set = compute_follow(kv.first);
        for (char symbol : follow_set) {
            cout << symbol << " ";
        }
        cout << "}" << endl;
    }
    return 0;
}
```

Output:

```
First(C) = { c }


...Program finished with exit code 0
Press ENTER to exit console.
```

Lab -7

Date: 21/02/2023

Aim: Consider the following grammar: S --> ABC A--> abA | ab B--> b | BC C--> c | cC Following any suitable parsing technique (prefer topdown), design a parser which accepts a string and tells whether the string is accepted by above grammar or not

Code:

```
#include <iostream>

#include <string>

using namespace std;

bool parseS(string input);

bool parseA(string input);

bool parseB(string input);

bool parseC(string input);

int main() {

   string input;

   cout << "Enter a string to parse: ";

   cin >> input;

   if (parseS(input)) {

      cout << "Accepted!" << endl;

   } else {

      cout << "Not accepted." << endl;

   }

   return 0;

}


// Parse the S non-terminal

bool parseS(string input) {

   if (input.empty()) {

      // Input string is empty

      return false;
```

P V S Jaswanth Kumar
A2305220687
6CSE-2 Y

```
    }
    if (input[0] == 'a') {
        // Try to parse an A production
        return parseA(input.substr(1));
    } else {
        // Invalid input
        return false;
    }
}
// Parse the A non-terminal
bool parseA(string input) {
    if (input.empty()) {
        // Input string is empty
        return false;
    }
    if (input[0] == 'a') {
        // Try to parse an abA production
        if (input.length() >= 3 && input[1] == 'b' && parseA(input.substr(2))) {
            return true;
        } else {
            return false;
        }
    } else if (input[0] == 'b') {
        // Try to parse a b production
        return parseB(input.substr(1));
    } else {
        // Invalid input
        return false;
    }
}
// Parse the B non-terminal
```

```
bool parseB(string input) {
    if (input.empty()) {
        // Input string is empty
        return false;
    }
    if (input[0] == 'b') {
        // Try to parse a b production
        return parseB(input.substr(1));
    } else if (input[0] == 'c') {
        // Try to parse a BC production
        if (parseC(input.substr(1))) {
            return true;
        } else {
            return false;
        }
    } else {
        // Invalid input
        return false;
    }
}
// Parse the C non-terminal
bool parseC(string input) {
    if (input.empty()) {
        // Input string is empty
        return false;
    }
    if (input[0] == 'c') {
        // Try to parse a c production
        return parseC(input.substr(1));
    } else if (input[0] == 'a' || input[0] == 'b') {
        // Invalid input
```

```
        return false;
    } else {
        // Try to parse a cC production
        if (parseC(input.substr(1))) {
            return true;
        } else {
            return false;
        }
    }
}
```

Output:

```
Enter a string to parse: S->ABC A->abA|ab B->b|BcC C->c|cC
Not accepted.


...Program finished with exit code 0
Press ENTER to exit console.
```

P V S Jaswanth Kumar
A2305220687
6CSE-2 Y

LAB 8

Date: / /2023

Aim: To Compute LEADING () and TRAILING() of the desired Non-Terminal Element of given Grammar.

Code:

```
# Define the grammar rules as a dictionary of non-terminals mapped to their production rules
rules = {
    'S': ['aB', 'bA'],
    'A': ['aS', 'c'],
    'B': ['bS', 'd']
}

# Define the non-terminal element to compute the LEADING and TRAILING sets for
non_terminal = 'S'

# Compute the LEADING set for the non-terminal element
leading = set()
for non_term, prod_rules in rules.items():
    for rule in prod_rules:
        if rule.startswith(non_terminal):
            if len(rule) > 1:
                leading.add(rule[1])
            else:
                leading |= set(rules[non_term]) - set(rule)
```

P V S Jaswanth Kumar
A2305220687
6CSE-2 Y

```
# Compute the TRAILING set for the non-terminal element
trailing = set()
for non_term, prod_rules in rules.items():
    for rule in prod_rules:
        if rule.endswith(non_terminal):
            if len(rule) > 1:
                trailing.add(rule[-2])
            else:
                trailing |= set(rules[non_term]) - set(rule)

# Print the results
print(f"LEADING({non_terminal}): {leading}")
print(f"TRAILING({non_terminal}): {trailing}")
```

Output:

LEADING(S): set()

TRAILING(S): {'b', 'a'}

LAB -9

Date: / /2023

Aim: Write a program which accepts a regular grammar with no left recursion, and no null production rules, and then it accepts a string and reports whether the string is accepted by the grammar or not.

Code:

```
# Define the grammar rules

rules = {
    'S': ['AB', 'BC'],
    'A': ['a'],
    'B': ['b'],
    'C': ['c']
}


# Define the input string to test

input_str = 'abc'


# Check if the input string is accepted by the grammar

def is_accepted(input_str, rules):

# Start by adding the start symbol to the stack

    stack = ['S']
    # Loop through the input string
    for char in input_str:
        # Pop the top symbol from the stack
        symbol = stack.pop()
        # If the symbol is a non-terminal, replace it with its production rules
        if symbol in rules:
```

P V S Jaswanth Kumar
A2305220687
6CSE-2 Y

```
            for rule in reversed(rules[symbol]):
                # Add each production rule to the stack in reverse order
                stack.extend(list(rule)[::-1])
        # If the symbol is a terminal, it must match the current input character
        elif symbol == char:
            pass
        # If the symbol is neither a terminal nor a non-terminal, the input is invalid
        else:
            return False
    # If we've looped through the input string and the stack is empty, the input is valid
    return not stack

# Test the function with the example grammar and input string
if is_accepted(input_str, rules):
    print(f"The string '{input_str}' is accepted by the grammar.")
else:
    print(f"The string '{input_str}' is not accepted by the grammar.")
```

Output:

The string 'abc' is not accepted by the grammar.

LAB-10

Date: / /2023

Aim: Design a parser which accepts a mathematical expression (containing integers only). If the expression is valid, then evaluate the expression else report that the expression is invalid. [Note: Design first the Grammar and then implement using ShiftReduce parsing technique. Your program should generate an output file clearly showing each step of parsing/evaluation of the intermediate subexpressions.

Code:

```
def evaluate(production):
    op = production[1]
    left_operand = int(production[0])
    right_operand = int(production[2])
    if op == '+':
        return left_operand + right_operand
    elif op == '-':
        return left_operand - right_operand
    elif op == '*':
        return left_operand * right_operand
    elif op == '/':
        return left_operand // right_operand


def parse_expression(expression):
    stack = []
    output = []
    i = 0
    while i < len(expression):
        if expression[i].isdigit():
            j = i + 1
            while j < len(expression) and expression[j].isdigit():
```

```
            j += 1
        output.append(expression[i:j])
        i = j
    elif expression[i] in ['+', '-', '*', '/']:
        while len(stack) > 0 and stack[-1] in ['*', '/']:
            output.append(stack.pop())
        stack.append(expression[i])
        i += 1
    elif expression[i] == '(':
        stack.append(expression[i])
        i += 1
    elif expression[i] == ')':
        while len(stack) > 0 and stack[-1] != '(':
            output.append(stack.pop())
        if len(stack) == 0:
            raise ValueError('Invalid expression')
        stack.pop()
        i += 1
    elif expression[i] == ' ':
        i += 1
    else:
        raise ValueError('Invalid expression')
while len(stack) > 0:
    if stack[-1] == '(':
        raise ValueError('Invalid expression')
    output.append(stack.pop())
stack = []
for token in output:
    if token.isdigit():
        stack.append(token)
```

```
        else:
            if len(stack) < 2:
                raise ValueError('Invalid expression')
            right_operand = stack.pop()
            left_operand = stack.pop()
            result = evaluate((left_operand, token, right_operand))
            stack.append(str(result))
    if len(stack) != 1:
        raise ValueError('Invalid expression')
    return int(stack[0])


expression = input('Enter a mathematical expression: ')
try:
    result = parse_expression(expression)
    print(f'{expression} = {result}')
except ValueError as e:
    print(e)
```

Output:

Enter a mathematical expression: ^^? E= ->E+T|E-T|T , T_ ->T*F|T/F|F,F->(E)|-F|+F\num F\num |num

Invalid expression